Imię i nazwisko	Wiktoria Sakowska (274931)
Nazwa uczelni	Uniwersytet Gdański
Nazwa wydziału	Matematyki, Fizyki i Informatyki
Kierunek	Informatyka Ogólnoakademicka
Prowadząca	mgr Laura Grzonka
Nazwa ćwiczenia	Projekt 1
Data zajęć	04.03.2025 r.
Data oddania	25.03.2025 r.

Co mam w lodówce?

Dokumentacja projektu

Programowanie w języku Python — Projekt I

Spis treści

1 Opis projektu **3**

2 Wymagania i instalacja **3**
2.1 Wymagania systemowe 3
2.2 Instalacja krok po kroku 3
2.3 Dostępne adresy 4

3 Architektura projektu **4**
3.1 Struktura katalogów 4
3.2 Aplikacje i ich odpowiedzialności 5
3.3 Stack technologiczny 5

4 Modele danych **5**
4.1 Diagram relacji 5
4.2 Opis modeli 6

5 Kluczowe funkcjonalności **6**
5.1 Wyszukiwarka po składnikach 6
5.2 System rekomendacji 7
5.3 Asystent kucharski AI 7
5.4 Konta użytkowników 7
5.5 Komentarze z ocenami 7
5.6 CRUD przepisów 8
5.7 REST API i Swagger 8

6 Management commands **8**

7 Testy **8**

8 Spełnienie wymagań projektowych **9**
8.1 Elementy niezbędne 9
8.2 Opcje dodatkowe (6 z wymaganych 3) 9

9 Podział pracy **10**
9.1 Praca samodzielna 10
9.2 Z pomocą AI 10
9.3 Z dokumentacji i samouczków 10

10 Bibliografia **10**
10.1 Sztuczna inteligencja 10
10.2 Frameworki i biblioteki 11
10.3 Dokumentacja i źródła internetowe 11

1 Opis projektu

„Co mam w lodówce?” to aplikacja webowa stworzona w frameworku Django, która rozwiązuje codzienny problem: użytkownik ma składniki w lodówce, ale nie wie co z nich ugotować.

Główne założenia aplikacji:

- Użytkownik zaznacza składniki, które posiada — aplikacja wyszukuje pasujące przepisy i sortuje je wg stopnia dopasowania.
- Przepisy można przeglądać, dodawać, edytować i usuwać (pełny CRUD).
- System rekomendacji podpowiada dania na podstawie preferencji dietetycznych użytkownika.
- Asystent kucharski AI (Google Gemini) generuje nowe przepisy z podanych składników.
- REST API z dokumentacją Swagger UI umożliwia testowanie endpointów w przeglądarce i Postmanie.

2 Wymagania i instalacja

2.1 Wymagania systemowe

- Python 3.10 lub nowszy
- pip
- Git

2.2 Instalacja krok po kroku

```
# 1. Klonowanie repozytorium
git clone <URL_REPOZYTORIUM>
cd <NAZWA_FOLDERU>
git checkout Projekt

# 2. Srodowisko wirtualne
python3 -m venv venv
source venv/bin/activate

# 3. Instalacja zaleznosci
pip install -r requirements.txt

# 4. Konfiguracja klucza API (opcjonalne)
echo "GEMINI_API_KEY=twoj_klucz" > .env

# 5. Migracje bazy danych
python manage.py migrate

# 6. Zaladowanie przykladowych danych
python manage.py seed_data
```

```
# 7. Uruchomienie serwera
python manage.py runserver
```

2.3 Dostępne adresy

Adres	Opis
`/`	Strona główna
`/admin/`	Panel administratora
`/przepisy/`	Lista przepisów
`/przepisy/szukaj/`	Wyszukiwarka po składnikach
`/przepisy/rekomendacje/`	Rekomendacje wg preferencji
`/przepisy/dodaj/`	Dodawanie przepisu
`/konto/rejestracja/`	Rejestracja
`/konto/logowanie/`	Logowanie
`/konto/profil/`	Profil użytkownika
`/asystent/`	Asystent kucharski AI
`/api/`	REST API (przeglądarka)
`/api/docs/`	Swagger UI
`/api/redoc/`	ReDoc

3 Architektura projektu

3.1 Struktura katalogów

Projekt składa się z 5 aplikacji Django i modułu konfiguracyjnego:

```
Project01/
|-- manage.py
|-- requirements.txt
|-- README.md, SOURCES.md, LICENSE
|-- .env, .gitignore
|-- data/                  # Dane do seedowania
|   |-- seed_data.json
|   `-- ingredients.csv
|-- fridge_project/        # Konfiguracja Django
|   |-- settings.py
|   |-- urls.py
|   `-- ...
|-- recipes/               # Glowna aplikacja
|-- accounts/              # Uzytkownicy i profile
|-- comments/              # Komentarze
|-- ai_assistant/          # Asystent AI
|-- api/                   # REST API + Swagger
```

```
|-- templates/              # Szablony globalne
|-- static/css/             # Wlasne style CSS
`-- media/                  # Przeslane zdjecia
```

3.2 Aplikacje i ich odpowiedzialności

Aplikacja	Odpowiedzialność
`recipes`	Przepisy, CRUD, wyszukiwarka po składnikach, rekomendacje, management commands
`accounts`	Rejestracja, logowanie, wylogowanie, profil użytkownika z preferencjami dietetycznymi
`comments`	Komentarze z oceną (1–5 gwiazdek) pod przepisami
`ai_assistant`	Asystent kucharski AI oparty na Google Gemini
`api`	REST API (Django REST Framework) z dokumentacją Swagger UI i ReDoc

3.3 Stack technologiczny

Technologia	Wersja	Zastosowanie
Python	3.10+	Język programowania
Django	5.x	Framework webowy
Django REST Framework	3.x	REST API
drf-spectacular	0.x	Dokumentacja Swagger/OpenAPI
Bootstrap	5.3.3	Framework CSS (CDN)
Bootstrap Icons	1.11.3	Ikony (CDN)
SQLite	3	Baza danych
Google Gemini API	gemini-2.0-flash	Asystent kucharski AI
Pillow	10.x	Obsługa przesyłania zdjęć
python-dotenv	—	Zmienne środowiskowe (.env)

4 Modele danych

Projekt wykorzystuje **7 modeli** z następującymi typami relacji: OneToOneField, ForeignKey, ManyToManyField oraz tabela pośrednia (through).

4.1 Diagram relacji

```
User ---- 1:1 ----> UserProfile ---- M:M ----> DietaryPreference
  |                       |                              |
  | 1:M                   | M:M (ulubione)               | M:M
```

```
  v                        v                                v
Recipe <-------- FavoriteRecipes                Recipe.dietary_prefs
  |
  |---- M:1 ----> Category
  |---- M:M ----> Ingredient (through RecipeIngredient)
  `---- 1:M ----> Comment
```

4.2 Opis modeli

Model	Kluczowe pola	Relacje
`Category`	name, slug, description	1:M → Recipe
`Ingredient`	name, unit (choices)	M:M → Recipe (through)
`Recipe`	title, slug, description, instructions, prep_time, difficulty, image, is_published	FK → User, FK → Category, M:M → Ingredient, M:M → DietaryPreference
`RecipeIngredient`	amount	FK → Recipe, FK → Ingredient
`UserProfile`	bio	1:1 → User, M:M → DietaryPreference, M:M → Recipe (ulubione)
`DietaryPreference`	name	M:M → UserProfile, M:M → Recipe
`Comment`	content, rating (1–5), created_at	FK → Recipe, FK → User

Tabela pośrednia `RecipeIngredient` (parametr `through`) pozwala przechowywać ilość danego składnika w przepisie — jest to rozwiązanie wymagające dodatkowej wiedzy o relacjach ManyToMany w Django.

5 Kluczowe funkcjonalności

5.1 Wyszukiwarka po składnikach

Główna funkcja aplikacji — użytkownik zaznacza składniki, które posiada, a system zwraca przepisy posortowane wg stopnia dopasowania.

Fragment logiki wyszukiwania (`recipes/views.py`):

```python
recipes = (
    Recipe.objects
    .filter(
        is_published=True,
        ingredients__in=selected_ingredients
    )
    .annotate(
        matched=Count(
            'ingredients',
            filter=models.Q(
```

```
                    ingredients__in=selected_ingredients
                )
            ),
            total_ingredients=Count('recipe_ingredients'),
        )
        .order_by('-matched')
        .distinct()
)
```

Wyjaśnienie:

- `filter()` — filtruje przepisy zawierające którykolwiek z wybranych składników.
- `annotate(matched=Count(...))` — dodaje pole `matched` zliczające ile składników przepisu pasuje do wybranych.
- `order_by('-matched')` — najlepsze dopasowanie na górze.
- `distinct()` — eliminuje duplikaty z relacji ManyToMany.

5.2 System rekomendacji

Rekomendacje opierają się na preferencjach dietetycznych zapisanych w profilu użytkownika. Widok `recipe_recommendations` pobiera preferencje z `UserProfile`, a następnie filtruje przepisy, które mają przypisane pasujące preferencje, sortując je wg liczby dopasowań.

5.3 Asystent kucharski AI

Integracja z Google Gemini API (`gemini-2.0-flash`):

```
genai.configure(api_key=settings.GEMINI_API_KEY)
model = genai.GenerativeModel('gemini-2.0-flash')
prompt = f"""Jestes asystentem kucharskim...
  Uzytkownik napisal: {user_message}"""
result = model.generate_content(prompt)
```

Klucz API przechowywany jest w pliku `.env` (nie w repozytorium) i ładowany przez `python-dotenv`.

5.4 Konta użytkowników

System kont obejmuje:

- Rejestracja (formularz `RegisterForm` rozszerzający `UserCreationForm`)
- Logowanie/wylogowanie (wbudowane widoki Django: `LoginView`, `LogoutView`)
- Profil użytkownika z preferencjami dietetycznymi (model `UserProfile`)
- Panel administratora Django (`/admin/`)

5.5 Komentarze z ocenami

Pod każdym przepisem użytkownicy mogą dodawać komentarze z oceną 1–5 gwiazdek. Ocena walidowana jest za pomocą `MinValueValidator` i `MaxValueValidator`.

5.6 CRUD przepisów

Pełny cykl życia przepisu:

- **Create** — formularz z formsetem składników (`inlineformset_factory`)
- **Read** — lista z filtrowaniem po kategorii, strona szczegółów
- **Update** — edycja z zachowaniem składników
- **Delete** — potwierdzenie usunięcia, dostępne tylko dla autora

5.7 REST API i Swagger

Aplikacja udostępnia REST API zbudowane z Django REST Framework:

Metoda	Endpoint	Opis
GET	`/api/przepisy/`	Lista przepisów
GET	`/api/przepisy/{slug}/`	Szczegóły ze składnikami i komentarzami
GET	`/api/skladniki/`	Lista składników
GET	`/api/kategorie/`	Lista kategorii
GET	`/api/szukaj/?ingredients=1&2`	Wyszukiwanie po składnikach
GET	`/api/docs/`	Swagger UI
GET	`/api/redoc/`	ReDoc

Schemat OpenAPI z `/api/schema/` można zaimportować do Postmana.

6 Management commands

Projekt zawiera 3 własne komendy `manage.py`:

Komenda	Opis	Opcje
`seed_data`	Seedowanie bazy danymi z `seed_data.json` i `ingredients.csv`	`--clear` — wyczyść bazę przed importem
`import_ingredients`	Import składników z pliku CSV	`--file sciezka` — wskaż inny plik
`cleanup_recipes`	Usuwanie nieopublikowanych przepisów starszych niż X dni	`--days 7`, `--dry-run` — podgląd bez usuwania

7 Testy

Projekt zawiera **34 testy jednostkowe** uruchamiane komendą:

```
python manage.py test
python manage.py test --verbosity=2
```

Obszar	Testowane elementy	Liczba
Modele `recipes`	Category, Ingredient, Recipe, RecipeIngredient	7
Modele `accounts`	UserProfile, DietaryPreference	2
Modele `comments`	Comment	2
Formularze	RecipeForm, IngredientSearchForm — walidacja	4
Widoki `recipes`	Strona główna, lista, szczegóły, tworzenie, wyszukiwarka	11
Widoki `accounts`	Rejestracja, logowanie	4
Widoki `comments`	Dodawanie komentarza, autoryzacja	4
Razem		**34**

8 Spełnienie wymagań projektowych

8.1 Elementy niezbędne

Wymaganie	Realizacja
Min. 5 podstron	10+ podstron
Komunikacja z bazą (CRUD)	Pełny CRUD przepisów
Widoki FBV lub CBV	Function-based views + ViewSety (API)
Min. 5 modeli z relacjami	7 modeli: 1:1, FK, M:M, through
Min. 1 formularz	7+ formularzy + formset
Obsługa 404 i 500	Własne szablony
Min. 3 management commands	seed_data, import_ingredients, cleanup_recipes
Testy	34 testy jednostkowe

8.2 Opcje dodatkowe (6 z wymaganych 3)

1. **Stylizacja CSS / framework** — Bootstrap 5.3 + Bootstrap Icons + własny plik `style.css`
2. **Konta użytkowników + logowanie** — rejestracja, logowanie, wylogowanie, profil, panel admina
3. **Komentarze** — z oceną 1–5 gwiazdek pod przepisami
4. **Przeszukiwanie strony** — wyszukiwarka przepisów po składnikach z rankingiem dopasowania
5. **Rekomendacje wg preferencji** — na podstawie preferencji dietetycznych z profilu
6. **Chat AI** — asystent kucharski oparty na Google Gemini

Dodatkowo: REST API z dokumentacją Swagger UI (poza listą wymaganych opcji).

9 Podział pracy

9.1 Praca samodzielna

Poniższe elementy zostały zaprojektowane i zaimplementowane przeze mnie samodzielnie:

- Koncepcja i projekt aplikacji — wybór tematu, zaplanowanie funkcjonalności
- Projektowanie modeli danych i relacji między nimi
- Logika wyszukiwarki po składnikach (`annotate`, `Count`, `Q objects`)
- System rekomendacji wg preferencji dietetycznych
- Formularze z formsetami (`inlineformset_factory`)
- Management commands z opcjami CLI (`--clear`, `--dry-run`, `--file`)
- Konfiguracja REST API (ViewSety, serializery, router)
- Testy jednostkowe
- Szablony HTML i dostosowanie stylów CSS
- Konfiguracja projektu Django i integracja komponentów
- Dokumentacja (README, SOURCES, niniejszy dokument)

9.2 Z pomocą AI

- Wsparcie przy pisaniu kodu — generowanie szkieletu widoków, szablonów, formularzy
- Debugowanie błędów
- Generowanie danych seedowych (przepisy, składniki w plikach JSON/CSV)
- Pomoc przy strukturyzacji projektu i planowaniu commitów

9.3 Z dokumentacji i samouczków

- Oficjalna dokumentacja Django — widoki, modele, formularze, testy
- Dokumentacja Django REST Framework — serializery, ViewSety
- Dokumentacja Bootstrap 5 — klasy CSS, komponenty
- Dokumentacja Google Gemini API — integracja modelu AI

10 Bibliografia

10.1 Sztuczna inteligencja

Narzędzie	**Model**	**Firma**	**Zastosowanie**	**Dostęp**
Claude	Claude Opus 4.6	Anthropic	Wsparcie przy kodowaniu, debugowanie, dane testowe	III 2025
Gemini	gemini-2.0-flash	Google	Asystent kucharski AI (w aplikacji)	III 2025

10.2 Frameworki i biblioteki

Nazwa	Wersja	Strona	Dostęp
Django	5.x	https://www.djangoproject.com/	III 2025
DRF	3.x	https://www.django-rest-framework.org/	III 2025
drf-spectacular	0.x	https://drf-spectacular.readthedocs.io/	III 2025
Bootstrap	5.3.3	https://getbootstrap.com/	III 2025
Bootstrap Icons	1.11.3	https://icons.getbootstrap.com/	III 2025
Pillow	10.x	https://pillow.readthedocs.io/	III 2025
google-generativeai	—	https://pypi.org/project/google-generativeai/	III 2025
python-dotenv	—	https://pypi.org/project/python-dotenv/	III 2025

10.3 Dokumentacja i źródła internetowe

Tytuł	URL	Dostęp
Django Documentation — Models	https://docs.djangoproject.com/en/5.1/topics/db/models/	III 2025
Django Documentation — Making queries	https://docs.djangoproject.com/en/5.1/topics/db/queries/	III 2025
Django Documentation — Writing views	https://docs.djangoproject.com/en/5.1/topics/http/views/	III 2025
Django Documentation — Forms	https://docs.djangoproject.com/en/5.1/topics/forms/	III 2025
Django Documentation — Formsets	https://docs.djangoproject.com/en/5.1/topics/forms/formsets/	III 2025
Django Documentation — Authentication	https://docs.djangoproject.com/en/5.1/topics/auth/	III 2025
Django Documentation — Testing	https://docs.djangoproject.com/en/5.1/topics/testing/	III 2025
Django Documentation — Custom management commands	https://docs.djangoproject.com/en/5.1/howto/custom-management-commands/	III 2025
DRF — Serializers	https://www.django-rest-framework.org/api-guide/serializers/	III 2025
DRF — ViewSets	https://www.django-rest-framework.org/api-guide/viewsets/	III 2025
drf-spectacular — Settings	https://drf-spectacular.readthedocs.io/en/latest/settings.html	III 2025
Google Gemini API — Quickstart	https://ai.google.dev/gemini-api/docs/quickstart	III 2025

Tytuł	URL	Dostęp
OpenAPI Specification	https://swagger.io/specification/	III 2025